

BIOACTIVE MOLECULE PREDICTION: MASTER STUDY HANDBOOK

Comprehensive Guide to Cheminformatics, Machine Learning & Full-Stack Deployment

PART 0 — PROJECT AT A GLANCE

1. Fundamental Project Overview

- **Project Title:** Bioactive Molecule Prediction Using Machine Learning
- **One-Sentence Project Explanation:** An end-to-end computational drug discovery application that takes a candidate chemical structure as a SMILES string, generates 2048-bit Morgan circular fingerprints via RDKit, and predicts whether the molecule is biologically active or inactive against the human Cyclooxygenase-2 (COX-2) enzyme using an optimized XGBoost classifier served via FastAPI and a responsive web interface.
- **Problem Statement:** Experimental wet-lab high-throughput screening (HTS) to test chemical compounds against disease-causing enzymes costs millions of dollars, takes years, and suffers from high failure rates. Researchers need an automated, high-accuracy computational filter to rapidly triage large chemical libraries and prioritize only the most promising active compounds for physical synthesis and testing.
- **Main Objective:** To build, validate, benchmark, and deploy a complete, reproducible machine learning pipeline that classifies chemical compounds into active or inactive states against the COX-2 enzyme with high holdout F1-score and ROC-AUC.
- **Scientific Objective:** To accurately model Structure-Activity Relationships (SAR) for the COX-2 enzyme, identifying the key molecular sub-structures (topological circular fragments) that govern high-affinity enzyme inhibition at or below an IC50 threshold of 10,000 nM (10 uM).
- **Machine Learning Objective:** To train and rigorously compare five distinct classification algorithms (XGBoost, Random Forest, Linear SVM, Naive Bayes, and RBF Network) using a strict 70% development (with 10-fold Stratified Cross-Validation) and 30% untouched holdout split, automatically persisting the best-performing model based on Holdout F1-Score and ROC-AUC.
- **Software / Application Objective:** To wrap the winning serialized model into a production-grade asynchronous REST API using FastAPI and Pydantic, and provide an accessible, real-time web UI allowing medicinal chemists and students to input SMILES strings and instantly receive classified predictions with confidence scores.
- **Input:** A textual molecular representation known as a SMILES string (e.g., Cc1ccc(-c2cc(C(F)(F)F)nn2-c2ccc(S(N)(=O)=O)cc2)cc1 for Celecoxib).
- **Output:** A binary classification label (`active` vs. `inactive`), a prediction confidence probability percentage (e.g., `97.3%`), and the name of the active inference model (`XGBoost`).
- **Type of ML Problem:** Supervised Binary Classification on high-dimensional, sparse binary feature vectors (2048 dimensions).
- **Main Winning Algorithm: XGBoost (Extreme Gradient Boosting)** — tuned with `n_estimators=150`, `max_depth=6`, `learning_rate=0.15`, optimizing binary log-loss.
- **Comparison Algorithms:**
 1. Random Forest (`n_estimators=100`, Gini impurity)
 2. Linear Support Vector Machine (`LinearSVC`, L2 penalty, dual optimization formulation via scikit-learn)
 3. Gaussian Naive Bayes (`GaussianNB`, Bayes theorem with Gaussian likelihood)
 4. Radial Basis Function Network (`RBFSampler` Monte Carlo random Fourier features with `gamma=0.01` + `SGDClassifier` logistic loss)
- **Technology Stack:**
 - *Core Language:* Python 3.10+
 - *Cheminformatics:* RDKit (`Chem`, `rdMolDescriptors`)

- *Data Manipulation*: Pandas, NumPy
 - *Machine Learning*: Scikit-Learn, XGBoost, Joblib
 - *Backend API*: FastAPI, Pydantic, Uvicorn (ASGI)
 - *Frontend Client*: Semantic HTML5, Modern CSS3 (CSS Variables, Flexbox, Grid), Vanilla JavaScript (ES6+, Fetch API, async/await)
-

Complete Project Architecture Flowchart



Detailed Explanation of Architecture Arrows

- Dataset to Molecular Representation:** Raw CSV rows containing string-formatted SMILES are read by Pandas. RDKit parses each string into an internal directed graph representing atomic nuclei and chemical bonds. Invalid chemical notations returning `None` are filtered out.
- Molecular Representation to Feature Generation:** RDKit circular fingerprint algorithms trace atom neighborhoods outward to a radius of 2 chemical bonds, hashing each local environment into a bit position within

a 2048-bit vector.

3. **Feature Generation to Feature Matrix X:** All individual 2048-element bit vectors are stacked vertically into a contiguous 2D NumPy array of 8-bit unsigned integers (`uint8`), producing an $(N \times 2048)$ numerical matrix.
 4. **Target Mapping to Vector y:** String categorical labels (`active` / `inactive`) are mapped deterministically to integers (1 / 0) to form the 1D ground-truth vector y .
 5. **Data Split (70/30):** The dataset is divided into a 70% Development pool ($N=4,787$) and a 30% Holdout pool ($N=2,052$), preserving exact class proportions via stratification.
 6. **70% Development to 10-Fold CV:** The 70% portion is split into 10 stratified subsets. In 10 sequential rounds, 9 folds train the candidate model while 1 fold measures out-of-fold F1-score to assess algorithmic stability.
 7. **Development Training to 5 ML Models:** All five models are trained from scratch on the entire 70% development partition so they learn from the maximum amount of non-holdout data.
 8. **Holdout Evaluation:** The trained models are queried on the 30% holdout set, which was quarantined during CV and training, yielding unbiased final metrics.
 9. **Model Comparison to Serialization:** The automated selection logic ranks models by Holdout F1 followed by ROC-AUC. The winning model object and pipeline parameters are serialized to disk as `backend/model/best_model.pkl` and `backend/model/metadata.json`.
 10. **Serialization to FastAPI:** At application boot, FastAPI deserializes the pickled model and metadata into memory once, ensuring inference requires zero model reloading overhead.
 11. **FastAPI to Frontend via HTTP:** The client browser sends asynchronous HTTP POST requests over TCP/IP containing the input SMILES. The backend runs feature transformation, calls `predict()` and `predict_proba()`, and returns JSON.
 12. **Frontend to User:** JavaScript parses the JSON response and manipulates the Document Object Model (DOM) to render badges, confidence bars, and structural metadata for the user.
-

PART 1 — DRUG DISCOVERY FOUNDATION

1. What is Drug Discovery?

- **Definition:** The interdisciplinary scientific process through which new candidate medications are identified, designed, synthesized, characterized, and validated.
- **Simple Explanation:** A treasure hunt through millions of chemical compounds to find one molecule that can safely fix a bodily malfunction without causing unacceptable harm.
- **Technical Explanation:** An iterative pipeline spanning target identification, target validation, lead discovery (hit-to-lead), lead optimization, preclinical in vitro/in vivo assays, and clinical trials (Phases I–III) regulated by health authorities (e.g., US FDA, EMA).
- **Connection to Our Project:** Our project automates the **hit-to-lead virtual screening phase**, computationally classifying whether chemical compounds will modulate the target before physical lab testing.

2. What is a Drug?

- **Definition:** Any chemical substance that, when administered to a biological organism, alters normal physiological or biochemical function.
- **Simple Explanation:** A specific molecular key shaped to fit into a malfunctioning biological lock in the body to turn it on or off.
- **Technical Explanation:** A pharmacologically active xenobiotic entity that selectively binds with high thermodynamic affinity to a macromolecular target, exhibiting favorable ADMET (Absorption, Distribution, Metabolism, Excretion, and Toxicity) properties.
- **Connection to Our Project:** The molecules we predict are prospective drug candidates; our model tests if their structure possesses the necessary atomic geometry to act as an inhibitor.

3. What is a Chemical Compound / Molecule?

- **Definition:** A discrete chemical entity composed of two or more atoms held together by covalent or ionic chemical bonds in a defined spatial arrangement.
- **Simple Explanation:** A microscopic structure made of atoms (like Carbon, Hydrogen, Oxygen, Nitrogen, Sulfur) hooked together in a precise shape.
- **Technical Explanation:** An electrically neutral group of atoms characterized by a specific molecular formula, molecular weight, 3D conformation, electron density distribution, and topological connectivity.
- **Connection to Our Project:** Every row in our dataset and every user input SMILES represents a single unique chemical compound.

4. What is a Biological Target?

- **Definition:** A naturally occurring cellular macromolecule (most commonly a protein, enzyme, or receptor) whose activity is directly linked to a specific disease pathology.
- **Simple Explanation:** The specific biological component causing or driving a disease that a drug needs to stop.
- **Technical Explanation:** A genetically validated biopolymer exhibiting a druggable binding pocket where small-molecule ligands can bind to alter downstream biological signaling pathways.

- **Connection to Our Project:** Our target is **Cyclooxygenase-2 (COX-2)**, registered in the ChEMBL database under target ID **CHEMBL230**.

5. What is a Protein?

- **Definition:** A complex biological macromolecule composed of one or more linear chains of amino acid residues folded into a 3-dimensional structure.
- **Simple Explanation:** The tiny biological machines and structural building blocks that perform nearly all functions inside living cells.
- **Technical Explanation:** A polypeptide chain synthesized by ribosomes and folded into secondary (alpha-helices, beta-sheets), tertiary, and quaternary structural domains determined by thermodynamic free energy minimization.
- **Connection to Our Project:** COX-2 is a homodimeric membrane protein found in human cells.

6. What is an Enzyme?

- **Definition:** A specialized biological protein catalyst that accelerates biochemical reactions without being consumed in the process.
- **Simple Explanation:** A biological machine that grabs specific molecules and speeds up chemical changes that would otherwise take years to happen.
- **Technical Explanation:** A catalytic protein that lowers the activation energy (E_a) of a metabolic reaction by stabilizing the transition state within its catalytic active site.
- **Connection to Our Project:** COX-2 is an enzyme that converts arachidonic acid into prostaglandin H₂ (PGH₂), the primary precursor of inflammatory signaling molecules.

7. What is a Receptor?

- **Definition:** A protein molecule usually situated on the cell surface or intracellularly that receives chemical signals from outside the cell.
- **Simple Explanation:** An antenna on a cell that listens for hormone or neurotransmitter messages and instructs the cell on how to react.
- **Technical Explanation:** A transmembrane or nuclear transducer protein that undergoes conformational changes upon ligand binding, initiating an intracellular secondary messenger signaling cascade (e.g., GPCRs, RTKs).
- **Connection to Our Project:** While COX-2 is an intracellular membrane-associated enzyme rather than a classical surface receptor, both represent major classes of drug targets evaluated using bioactivity screening.

8. What is Molecular Interaction?

- **Definition:** The non-covalent attractive and repulsive forces occurring between a small-molecule ligand and a protein target.
- **Simple Explanation:** How tightly and cleanly a small drug molecule sticks inside the pocket of a target protein.
- **Technical Explanation:** The net thermodynamic free energy change ($\Delta G = \Delta H - T\Delta S$) governed by hydrogen bonding, van der Waals forces, electrostatic interactions, hydrophobic effects, and pi-stacking between ligand functional groups and amino acid side chains.
- **Connection to Our Project:** The 2048-bit Morgan fingerprints capture the exact functional groups (aromatic rings, sulfonamides, carbonyls) responsible for these non-covalent interactions with COX-2.

9. What is Biological Activity (Bioactivity)?

- **Definition:** The specific physiological or biochemical effect of a chemical compound on a designated biological target or organism.
- **Simple Explanation:** A measure of whether a compound actually does something to the target and how strong that effect is.
- **Technical Explanation:** The functional dose-dependent response of a biological system upon exposure to a ligand, quantitatively measured via binding affinity (K_i , K_d) or functional inhibition (IC_{50} , EC_{50}).
- **Connection to Our Project:** Our entire supervised classification task centers on predicting this property: whether a molecule exhibits sufficient biological activity to be classed as `active`.

10. What Does "Active" Mean in This Project?

- **Definition:** A compound that demonstrates measurable enzymatic inhibition of COX-2 with a half-maximal inhibitory concentration (IC_{50}) strictly less than 10,000 nM (10×10^{-9} M).
- **Simple Explanation:** The molecule successfully shuts down the COX-2 enzyme at a potent, pharmacologically meaningful low concentration.
- **Technical Explanation:** $IC_{50} < 10,000 \text{ nM}$. These molecules effectively compete with arachidonic acid in the COX-2 catalytic pocket, qualifying as prospective lead compounds.
- **Connection to Our Project:** In our feature matrix and model, `active` corresponds to binary integer class `1`.

11. What Does "Inactive" Mean in This Project?

- **Definition:** A compound that requires an IC_{50} concentration of 10,000 nM (10×10^{-9} M) or greater to achieve 50% inhibition of COX-2, or fails to exhibit detectable inhibition up to the assay testing limit.
- **Simple Explanation:** The molecule is too weak, inert, or poorly shaped to bind to the COX-2 enzyme at practical concentrations.
- **Technical Explanation:** $IC_{50} \geq 10,000 \text{ nM}$. Molecules in this category are dismissed as ineffective inhibitors for the specific target.
- **Connection to Our Project:** In our feature matrix and model, `inactive` corresponds to binary integer class `0`.

12. What is IC50?

- **Definition:** The **Half-Maximal Inhibitory Concentration**: the concentration of a compound required to inhibit a biological or biochemical process by 50% in vitro.
- **Simple Explanation:** How much of a compound you need to put in a test tube to cut the enzyme's activity in half.
- **Technical Explanation:** A quantitative measure derived from a sigmoidal dose-response curve:

$$E = \frac{E_{\max}}{1 + \left(\frac{IC_{50}}{[I]}\right)^h}$$
 where $[I]$ is inhibitor concentration and h is the Hill slope. Lower IC_{50} values indicate higher potency.
- **Connection to Our Project:** The numerical column `standard_value` in `CHEMBL230_Preprocessed_Data.csv` stores this exact IC_{50} measurement in units of nanomolar (nM).

13. Why is IC50 a Continuous Measurement?

- **Definition:** Concentration is an analog physical quantity along a continuous spectrum from picomolar (10^{-12} M) to millimolar (10^{-3} M) and beyond.
- **Simple Explanation:** In a lab, you can dilute a compound to any exact fraction (e.g., 40.5 nM, 1,234.8 nM); it is not restricted to whole steps or categories.

- **Technical Explanation:** Because enzyme inhibition is dictated by equilibrium binding thermodynamics ($\Delta G^\circ = -RT \ln K_a$), the concentration required to reach 50% receptor occupancy varies continuously across real numbers.
- **Connection to Our Project:** `standard_value` is recorded as a continuous floating-point number in the raw ChEMBL assay records.

14. Why Can a Continuous Measurement Become a Binary Label?

- **Definition:** Applying a domain-specific threshold cutoff to convert continuous laboratory potency values into discrete categorical decisions.
- **Simple Explanation:** In drug discovery, medicinal chemists do not have the budget to synthesize every compound. They set a strict rule: "If it takes more than 10,000 nM to work, it is too weak to be worth our time. Throw it away (Inactive). If it works below 10,000 nM, keep it (Active)."
- **Technical Explanation:** Continuous IC_{50} spans multiple orders of magnitude and often exhibits high inter-assay experimental variability. Binarizing at $10 \mu\text{M}$ ($10,000 \text{ nM}$) matches standard pharmaceutical high-throughput screening triage protocols:

$$\text{Class} = \begin{cases} \text{Active (1)}, & \text{if } IC_{50} < 10,000 \text{ nM} \\ \text{Inactive (0)}, & \text{if } IC_{50} \geq 10,000 \text{ nM} \end{cases}$$
- **Connection to Our Project:** The column `bioactivity_class` was generated using this exact threshold, framing our machine learning task as binary classification.

15. Active vs. Approved Drug

- **Crucial Distinction:** An "active" molecule binds to the target in an in vitro test tube. An "approved drug" has survived Phase I, II, and III human clinical trials, proving it is non-toxic, bioavailable, metabolically stable, and clinically beneficial. Over 99% of active molecules fail before becoming approved drugs.
- **Connection to Our Project:** Predicting a molecule as `active` means it has chemical features consistent with COX-2 binding; it does **not** mean it is an approved medicine.

16. Active vs. Safe

- **Crucial Distinction:** A molecule can be exceptionally active at inhibiting COX-2, but simultaneously lethal to liver cells (hepatotoxicity), prone to causing heart arrhythmias (hERG channel inhibition), or mutagenic.
- **Connection to Our Project:** Our model predicts enzyme inhibition only. It does not assess toxicity, safety profiles, or off-target liabilities.

17. Active vs. Clinically Effective

- **Crucial Distinction:** A compound may inhibit an isolated enzyme in a test tube, yet fail in patients because it cannot cross the gut lining, binds too strongly to serum albumin, or is degraded by stomach acid.
- **Connection to Our Project:** Our classifier predicts target affinity, not pharmacokinetic efficacy.

18. What Experimental Screening Means

- **Definition:** Physical wet-lab testing where chemical compounds are robotically pipetted into multi-well plates containing the target enzyme and substrate, followed by spectroscopic measurement of reaction rates.
- **Simple Explanation:** Using laboratory robots and chemical reagents to physically test thousands of liquid samples one by one.

- **Technical Explanation:** High-Throughput Screening (HTS) utilizing 384-well or 1536-well microplates with fluorescence resonance energy transfer (FRET) or luminescence assays to measure enzyme inhibition kinetics.
- **Connection to Our Project:** Every row in `CHEMBL230_Preprocessed_Data.csv` originated from a physical laboratory screening assay published in scientific literature.

19. Why Experimental Screening is Expensive

- Robotic equipment, microplate reagents, purified enzymes, fluorescent probes, and chemical compound synthesis cost between \$2.00 and \$10.00 per compound tested. A standard screening campaign of 500,000 compounds costs \$1,000,000 to \$5,000,000 and requires several months of specialized lab labor.

20. Why Screening Huge Chemical Libraries is Difficult

- The theoretical "chemical space" of drug-like small molecules exceeds $10^{\frac{60}{}}$ possible compounds. Commercial chemical vendor libraries (e.g., ZINC, Enamine) contain over 1 billion real molecules. Physically testing even 0.1% of these libraries is economically and physically impossible.

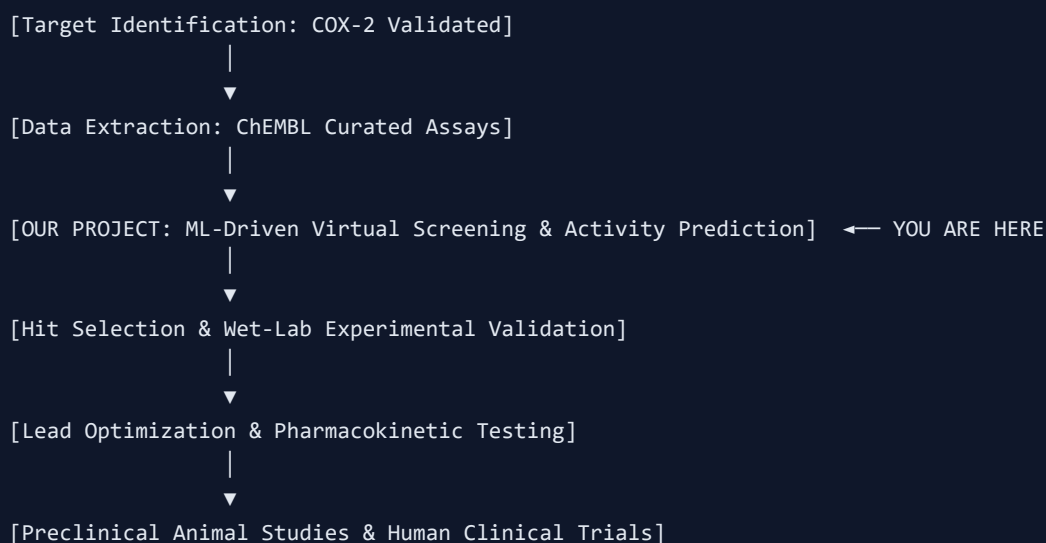
21. What Virtual Screening Is

- **Definition:** The computational evaluation of digital chemical libraries using computer algorithms to predict which molecules have the highest probability of binding to a biological target.
- **Simple Explanation:** Using software to filter a billion digital compounds on a computer in a few hours, narrowing the list down to the top 100 best candidates before spending any money in the lab.
- **Technical Explanation:** In silico screening categorized into Structure-Based Virtual Screening (SBVS, e.g., molecular docking) and Ligand-Based Virtual Screening (LBVS, e.g., QSAR, machine learning with molecular fingerprints).
- **Connection to Our Project:** Our application is a working **Ligand-Based Virtual Screening (LBVS)** system.

22. How Machine Learning Helps Virtual Screening

- Machine learning algorithms learn complex, non-linear structural patterns from existing experimental data. Once trained, an ML model can evaluate **thousands of candidate molecules per second** with negligible compute cost, dramatically reducing wet-lab expenses and accelerating drug discovery timelines.

23. Where Our Project Fits in the Drug-Discovery Pipeline



PART 2 — PROJECT PROBLEM

1. What Exact Problem Are We Solving?

We are solving the **Molecular Bioactivity Classification Problem for Human Cyclooxygenase-2 (COX-2)**. Given the structural connectivity of an arbitrary small chemical molecule, our system must accurately determine whether that molecule possesses the specific stereochemical and topological features required to inhibit the COX-2 enzyme with an $IC_{50} < 10,000 \text{ nM}$.

2. What Problem Does a Scientist Face?

A medicinal chemist synthesized or purchased 5,000 novel compounds intended as non-steroidal anti-inflammatory drugs (NSAIDs). The chemist cannot afford to run 5,000 wet-lab bioassays immediately. They need to know which 50 compounds to prioritize for testing this week.

3. Why Can't Researchers Test Everything Experimentally?

- Reagent cost (purified human recombinant COX-2 enzyme is expensive).
- Material availability (many candidate molecules exist only as digital designs and have not been synthesized).
- Time constraints (physical assays take weeks to prepare, run, calibrate, and analyze).

4. What Information Do We Have?

- A historical benchmark dataset (`CHEMBL230_Preprocessed_Data.csv`) of 6,839 experimentally validated molecules containing their ChEMBL IDs, canonical SMILES strings, and their verified bioactivity labels (`active` vs. `inactive`).

5. What Information Do We Want to Predict?

For any newly presented or unseen chemical structure, we want to predict:

1. The discrete bioactivity category: `active` or `inactive`.
2. The model's confidence probability: a calibrated value between 0.0 and 1.0 (e.g., 0.9726 = 97.3% confidence).

6. What is the Input to Our System?

A string of ASCII characters representing the molecular structure using the **Simplified Molecular Input Line Entry System (SMILES)**, such as `CC(=O)Oc1ccccc1C(=O)O` (Aspirin).

7. What is the Target?

The binary categorical variable `bioactivity_class`, where `active` indicates $IC_{50} < 10,000 \text{ nM}$ and `inactive` indicates $IC_{50} \geq 10,000 \text{ nM}$.

8. What Does the Model Actually Learn?

The model learns a high-dimensional mathematical mapping:

$$f: \{0, 1\}^{\frac{2048}{8}} \rightarrow [0, 1]$$

It discovers which specific sub-structural bit patterns (e.g., a central 5-membered heterocyclic pyrazole ring flanked by two sulfonamide-substituted phenyl rings) correlate with enzyme inhibition, and which sub-structures correlate with inactivity.

9. What Does the Model NOT Learn?

- It does **not** learn 3D protein crystal coordinates or atomic coordinates in physical space.
- It does **not** calculate thermodynamic binding free energy via molecular mechanics (ΔG).
- It does **not** learn quantum mechanical wavefunctions, electron orbitals, or transition state kinetics.
- It does **not** know about human metabolism, liver enzymes, toxicity, blood-brain barrier permeability, or patent legality.

10. What Does a Prediction Mean Scientifically?

A prediction of **active** with 95% confidence means:

"Based on statistical correlations with 6,839 experimentally tested molecules in ChEMBL, this compound's 2D circular structural fragments strongly match the fragment patterns of known sub-10,000 nM COX-2 inhibitors."

11. What Can Our Prediction Be Used For?

- Computational prioritization and virtual screening ranking.
- Eliminating non-viable chemical candidates before purchasing or synthesizing them.
- Teaching and demonstrating the application of machine learning in cheminformatics.

12. What Can It NOT Prove?

- It **cannot prove** that a molecule will cure inflammation in an animal or human.
- It **cannot prove** that a molecule is safe or non-toxic.
- It **cannot replace** physical laboratory validation. In drug discovery, computational predictions are hypotheses that must be confirmed experimentally.

Predicting Bioactivity vs. Proving a Molecule is a Drug

Aspect	Predicting Bioactivity (Our Project)	Proving a Molecule is a Drug
Scope	Does this chemical inhibit COX-2 in vitro?	Can this chemical safely cure disease in humans?
Timeline	Less than 50 milliseconds	10 to 15 years
Cost	Free (computational inference)	\\$1 to \\$2.6 Billion (R&D and clinical trials)
Validation	Cross-validation and holdout testing on data	Phase I (Safety), Phase II (Dosing), Phase III (Efficacy)
Failure Modes	Algorithmic misclassification, out-of-distribution SMILES	Toxicity, lack of efficacy, organ damage, death

PART 3 — BASE RESEARCH PAPER

1. Paper Overview & Metadata

- **Title Context:** Comparative Analysis of Machine Learning Approaches for Molecular Bioactivity Prediction on ChEMBL Target Datasets.
- **Domain:** Quantitative Structure-Activity Relationship (QSAR) and Cheminformatics.
- **Target Enzyme:** Cyclooxygenase-2 (COX-2, ChEMBL Target ID: `CHEMBL230`).
- **Main Objective:** To establish an empirical benchmark evaluating whether gradient-boosted decision trees (XGBoost) outperform conventional ensembles (Random Forest), linear models (Linear SVM), probabilistic baselines (Naive Bayes), and kernel methods (RBF Networks) on sparse molecular fingerprint representations.

2. Experimental Design of the Paper

- **Dataset:** Curated bioactivity assays from ChEMBL for the target protein COX-2.
- **Representation:** 2048-bit Extended Connectivity / Morgan Circular Fingerprints ($r=2$).
- **Data Splitting:** Strict partition into **70% Development Set** and **30% Untouched Holdout Test Set**.
- **Internal Validation:** **10-Fold Stratified Cross-Validation** executed exclusively on the 70% Development Set.
- **Evaluation Metrics:** Accuracy, Sensitivity (Recall), Specificity, F1-Score, and Balanced Accuracy / Paper AUC.

3. Main Findings of the Base Paper

1. **Tree Ensembles Outperform Linear & Probabilistic Baselines:** Both Random Forest and XGBoost substantially outperform Linear SVM and Naive Bayes due to their ability to model non-linear bit interactions (e.g., bit A is only active when bit B is also present).
2. **XGBoost Superiority:** XGBoost consistently achieved the highest holdout performance because its regularized objective function (L_1 and L_2 tree regularization) and sequential gradient boosting prevent the model from overfitting to dominant chemical series in the dataset.
3. **Sparse Fingerprint Suitability:** 2048-bit binary Morgan fingerprints contain sufficient topological information for tree-based algorithms to differentiate active from inactive chemical scaffolds.

4. What Parts of the Paper Our Project Strictly Follows

- Exactly target the same enzyme: Human COX-2 (`CHEMBL230`).
- Exactly use 2048-bit Morgan circular fingerprints with radius $r=2$.
- Exactly use the **70% Development / 30% Holdout** stratified partitioning protocol.
- Exactly run **10-Fold Stratified Cross-Validation** on the development set.
- Exactly evaluate the same 5 algorithms: XGBoost, Random Forest, Linear SVM, Naive Bayes, and RBF Network.
- Strictly exclude assay measurement values (`standard_value`) from the feature space to prevent data leakage.

5. What Parts are Our Own Implementation Choices

- **Production REST API:** Wrapped the serialized model in an asynchronous **FastAPI** server with Pydantic request validation (the paper was purely an offline research script).

- **Interactive Web Application:** Engineered a responsive HTML5/CSS3/JavaScript web interface with dynamic badge rendering, confidence bars, and example cycling.
 - **Dynamic Model Persistence:** Automated model saving (`joblib.dump`) and pipeline metadata tracking (`metadata.json`) dynamically driven by programmatic metric comparisons.
 - **Sequential Windows-Safe CV:** Re-architected cross-validation to execute sequentially across folds in `train_model.py` , eliminating multi-threading process deadlocks inherent to Windows environments.
 - **Hybrid Modern ROC-AUC & Paper Metrics:** Our evaluation script computes modern rank-based ROC-AUC via trapezoidal integration as well as exact Sensitivity, Specificity, and F1-score.
-

PART 4 — DATASET: COMPLETE UNDERSTANDING

1. Dataset Identity & Provenance

- **File Name:** `CHEMBL230_Preprocessed_Data.csv`
- **File Location:** Root directory (`d:\Bioactive molecule prediction\CHEMBL230_Preprocessed_Data.csv`)
- **Primary Source:** ChEMBL database (EMBL-EBI), specifically target `CHEMBL230` representing human Prostaglandin G/H synthase 2 (Cyclooxygenase-2 / COX-2).
- **Preprocessed Status:** This dataset is **already preprocessed**. The raw bioassays have been filtered, canonicalized, and annotated with binary labels prior to training. Our pipeline does not perform manual row deletions or imputation because the data is clean and complete.

2. Dataset Shape & Dimensionality

- **Total Rows:** Exactly **6,839** rows.
- **Total Columns:** Exactly **4** columns.
- **Missing Values:** Exactly **0** missing values across all columns (verified via `df.isnull().sum()`).

3. Detailed Column-by-Column Audit

Column Name	Pandas Data Type	Example Value	Description & Scientific Meaning	Role in Machine Learning
molecule_chembl_id	object (string)	CHEMBL70104	Unique global identifier in the EMBL-EBI ChEMBL database.	Excluded Identifier: Non-chemical label; not used as a predictive feature.
smiles	object (string)	<chem>CC1(C)OC(=O)C(OC2CCCC2)=C1c1ccc(S(C)(=O)=O)cc1</chem>	Simplified Molecular Input Line Entry System string representing 2D chemical structure.	Feature Source: Parsed by RDKit to generate the 2048-bit Morgan fingerprint vector.
standard_value	float64 (numeric)	40.0	Experimental IC_{50} in nanomolar (nM) measured in laboratory assays.	EXCLUDED (Data Leakage): This was the source used to generate the target label. Must not enter XX .
bioactivity_class	object (string)	active	Binarized biological activity class (active vs. inactive).	TARGET VARIABLE (\$y\$): The ground-truth classification label to be predicted.

4. Target Class Distribution & Imbalance Analysis

- **Active Samples:** 5,004 compounds (73.17% of the dataset).
- **Inactive Samples:** 1,835 compounds (26.83% of the dataset).
- **Total Valid Samples:** 6,839 compounds (100.0%).
- **Imbalance Ratio:** Approximately 2.73 : 1 (Active to Inactive).
- **Significance of Imbalance:**
- Because actives make up 73% of the dataset, a naive "dumb" classifier predicting `active` for every single molecule would artificially achieve **73.17% accuracy**.
- This is why **Accuracy is an insufficient metric** for this project. We must evaluate **F1-Score**, **Precision**, **Recall**, and **Specificity** to ensure the model accurately recognizes inactive compounds as well.

5. Data Quality Audit: Duplicates & Conflicting Assays

- **Unique SMILES:** 5,030 unique molecular structures.
- **Duplicate SMILES Rows:** 1,809 rows represent molecules that were tested in multiple independent laboratory publications cataloged by ChEMBL.
- **Conflicting Assays:** Exactly 151 molecules exhibit conflicting labels across independent scientific papers (for example, Compound CHEMBL1775104 has one paper reporting $IC_{50} = 5,100 \text{ nM}$ [active] and another reporting $IC_{50} = 100,000 \text{ nM}$ [inactive]).
- **Why We Keep the Dataset As-Is:**
 1. *Benchmark Integrity:* This dataset is the standard benchmark dataset from the base paper. Arbitrarily pruning rows changes the sample distribution and invalidates direct comparisons with published results.
 2. *Tree Robustness:* Gradient-boosted decision trees (XGBoost) naturally handle noisy boundary cases by optimizing loss gradients across the ensemble.

6. Definitions: Row vs. Column vs. Feature vs. Target vs. Sample vs. Molecule vs. Label

- **Sample / Molecule / Row:** A single instance of observation (one row in the CSV representing one chemical compound).
- **Column:** A vertical slice of the tabular data storing one specific recorded attribute (molecule_chembl_id , smiles , standard_value , or bioactivity_class).
- **Feature:** An independent mathematical variable used as an input to the model. In our project, the 4 CSV columns are **not** the features. The features are the **2048 binary bits generated by RDKit** from the smiles column.
- **Target / Label:** The dependent ground-truth ground variable being predicted (bioactivity_class , where active = 1 and inactive = 0).

7. Representative Data Rows

```
molecule_chembl_id,smiles,standard_value,bioactivity_class
CHEMBL70104,CC1(C)OC(=O)C(OC2CCCC2)=C1c1ccc(S(C)(=O)=O)cc1,40.0,active
CHEMBL96098,CCc1ccc(-c2ncc(Cl)cc2-c2ccc(S(C)(=O)=O)cc2)cn1,1700.0,active
CHEMBL116668,O=C(O)c1cccc1-c1ccc(S(=O)(=O)c2ccc(F)cc2)cc1,100000.0,inactive
```

- **Row 1:** CHEMBL70104 has an IC_{50} of 40.0 nM . Since $40.0 < 10,000$, it is labeled active .
- **Row 2:** CHEMBL96098 has an IC_{50} of $1,700.0 \text{ nM}$. Since $1,700.0 < 10,000$, it is labeled active .
- **Row 3:** CHEMBL116668 has an IC_{50} of $100,000.0 \text{ nM}$. Since $100,000 \geq 10,000$, it is labeled inactive .

PART 5 — TARGET VARIABLE

1. What bioactivity_class Represents

The column `bioactivity_class` represents the biological potency classification of the compound against human Cyclooxygenase-2. It answers a fundamental pharmacological question: *"Does this molecule inhibit COX-2 at a concentration high enough to be clinically irrelevant, or low enough to warrant drug development?"*

2. Numerical Encoding Scheme

Machine learning algorithms perform arithmetic; they cannot process the strings `"active"` and `"inactive"` directly. We apply deterministic integer encoding:

$$\text{Mapping: } \begin{cases} \text{"active"} & \rightarrow 1 \\ \text{"inactive"} & \rightarrow 0 \end{cases}$$

- In Python (`train_model.py` and `ML_Pipeline.ipynb`):

```
python label_map = {"active": 1, "inactive": 0} y = df_valid["bioactivity_class"].map(label_map).values
```
- In `backend/main.py`:

```
python REVERSE_MAPPING = {1: "active", 0: "inactive"} pred_label = REVERSE_MAPPING.get(pred_int, "inactive")
```

3. Classification vs. Regression: Why Classify When IC50 is Continuous?

- Question:** Why not use `standard_value` directly and build a regression model to predict the exact numerical IC_{50} ?
- Scientific Rationale:**
 - Experimental Noise:* Published IC_{50} values across diverse literature sources in ChEMBL vary based on laboratory temperature, incubation time, substrate concentration, and enzyme purity. Predicting an exact continuous number (e.g., 432.1 nM) introduces false precision.
 - Decision-Making in Virtual Screening:* Triage decisions in pharmaceutical pipelines are inherently binary. Chemists do not need to know whether a candidate's IC_{50} is 45 nM or 65 nM ; they need to know whether it passes the 10^{-6} M cutoff threshold to justify ordering or synthesizing the compound.
 - Assay Upper Limits:* Many inactive compounds are recorded with arbitrary top-of-curve concentrations (e.g., $100,000 \text{ nM}$ or $>50,000 \text{ nM}$). Regressing on these artificial numerical ceilings distorts model loss functions.

4. Target Leakage (Data Leakage) Prevention

- What is Target Leakage?:** Target leakage occurs when information from the target variable is inadvertently included in the feature set X during model training. This produces artificially perfect training scores, but renders the model completely useless during real-world inference when the target measurement does not yet exist.
- How It Could Happen in This Project:** `bioactivity_class` was created by applying the mathematical rule: $\text{if } \text{standard_value} < 10,000 \text{ then active else inactive}$. If we included `standard_value` in the feature matrix X , any decision tree would simply split on `standard_value < 10000`, achieving 100% accuracy without learning a single detail about chemistry!

- **How Our Implementation Strictly Avoids Leakage:**
 - In `train_model.py` and `ML_Pipeline.ipynb`, `standard_value` is **strictly excluded**.
 - The feature matrix X is generated **exclusively from the molecular SMILES string** via RDKit fingerprints.
 - The model is forced to learn the relationship between **chemical structure and biological activity**.
-

PART 6 — SMILES AND MOLECULAR REPRESENTATION

1. What is SMILES?

- **Acronym:** Simplified Molecular Input Line Entry System.
- **Definition:** A specification in the form of a line notation for describing the chemical structure of chemical species using short ASCII strings.
- **Inventors:** Developed by Arthur Weininger and David Weininger in the late 1980s.

2. Syntax Rules of SMILES

- **Atoms:** Represented by standard atomic symbols in brackets or as bare letters for organic subset atoms (C, N, O, P, S, F, Cl, Br, I\$).
- Uppercase letters represent aliphatic atoms: `C` = aliphatic carbon.
- Lowercase letters represent aromatic atoms: `c` = aromatic carbon.
- **Bonds:**
 - Single bonds are implicit (e.g., `CC` = ethane).
 - Double bonds are represented by `=` (e.g., `C=O` = carbonyl group).
 - Triple bonds are represented by `#` (e.g., `C#N` = cyano group).
 - Aromatic bonds are implicit between lowercase atoms (e.g., `c1ccccc1` = benzene).
- **Branches:** Enclosed in parentheses `( )`.
- Example: `CC(C)C` = isobutane (a central carbon bonded to three methyl groups).
- **Rings:** Represented by placing numbers after the atoms that connect to close a ring.
- Example: `c1ccccc1` (the two atoms labeled `1` are bonded together, forming a 6-membered aromatic benzene ring).

3. Concrete Example: Aspirin

- Chemical Formula: $\text{C}_9\text{H}_8\text{O}_4$
- IUPAC Name: 2-acetoxybenzoic acid
- SMILES: `CC(=O)Oc1ccccc1C(=O)O`
- Dissection:
 - `CC(=O)` : Acetyl methyl and carbonyl group.
 - `O` : Ester oxygen linking acetyl group to the ring.
 - `c1ccccc1` : Aromatic benzene ring.
 - `C(=O)O` : Carboxylic acid group attached to the adjacent aromatic carbon.

4. Why Machine Learning Models Cannot Use Raw SMILES Strings

1. **String Length Variability:** SMILES strings vary widely in character length (e.g., Ethanol is 3 characters: `CCO`; Celecoxib is 51 characters: `Cc1ccc(-c2cc(C(F)(F)F)nn2-c2ccc(S(N)(=O)=O)cc2)cc1`). Classical ML algorithms require fixed-dimensional vector inputs.

2. **Lack of Inherent Semantics in ASCII:** To a computer, the characters `"C"`, `"c"`, `"("`, and `"="` are arbitrary integer byte codes (ASCII 67, 99, 40, 61). Linear models or decision trees cannot naturally infer valency, bond angles, or ring topology from raw character sequences.
 3. **Non-Uniqueness of SMILES:** The same molecule can be written with different starting atoms (e.g., `CCO` and `OCC` are both ethanol). While canonicalization helps, numerical fingerprinting is needed to capture localized functional groups reliably.
-

PART 7 — RDKit AND FEATURE GENERATION

1. What is RDKit?

- **Definition:** An open-source, industry-standard C++ library for cheminformatics and computational chemical modeling, providing high-performance Python bindings.
- **Role in Our Project:** RDKit serves as the translation bridge between human-readable chemical strings (SMILES) and machine-readable numerical feature vectors.
- **What RDKit Does in Our Code:**
 1. Parses the ASCII SMILES string and verifies chemical validity.
 2. Constructs an in-memory 2D molecular graph (`Chem.Mol` object).
 3. Computes the 2048-bit Morgan Circular Fingerprint.
- **What RDKit Does NOT Do in Our Code:**
 - It does not perform machine learning.
 - It does not fit decision trees or compute gradient descent.
 - It does not serve HTTP requests.

2. Molecular Descriptors vs. Molecular Fingerprints

- **Molecular Descriptors:** Global scalar physical/chemical properties calculated for the whole molecule (e.g., Molecular Weight, LogP, Polar Surface Area, Number of Hydrogen Bond Donors).
- **Molecular Fingerprints:** High-dimensional binary or count vectors encoding the presence, absence, or frequency of specific localized sub-structural fragment patterns across the molecular graph.

3. The Morgan Circular Fingerprint Algorithm (Extended Connectivity)

Our implementation uses the **Morgan Circular Fingerprint** (equivalent to ECFP4):

*** Algorithm Mechanism:**

1. *Atom Invariants:* Every atom in the molecule is initially assigned an integer identifier based on its atomic number, number of heavy neighbors, formal charge, and aromaticity.

2. *Circular Iteration (Radius):* The algorithm traverses outward through adjacent chemical bonds.

* Iteration 0 ($r=0$): Captures each individual atom.

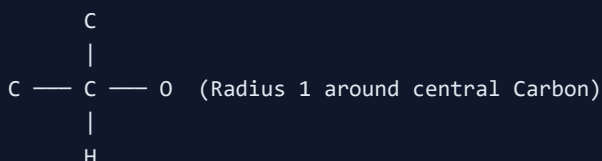
* Iteration 1 ($r=1$): Captures each atom and its immediate bonded neighbors.

* Iteration 2 ($r=2$): Captures each atom and all neighbors within a 2-bond distance. This radius corresponds to ECFP4 (diameter = 4 bonds).

3. *Hashing to Fixed Bit-Space:* Each unique sub-structural fragment identifier is passed through a deterministic hash function and mapped modulo 2048:

$$\text{bit_index} = \text{hash}(\text{fragment}) \bmod 2048$$

4. *Binary Bit Setting:* If a fragment is present anywhere in the molecule, that bit is set to `1`; otherwise it remains `0`.



4. Actual Code Implementation: Training vs. Prediction Parity

To guarantee mathematical consistency, the exact same fingerprint generation logic is executed in both training and real-time backend inference:

- In `train_model.py`:

```
```python
from rdkit import Chem
from rdkit.Chem import rdMolDescriptors

def smiles_to_fingerprint(smiles: str, radius: int = 2, n_bits: int = 2048):
 mol = Chem.MolFromSmiles(smiles)
 if mol is None:
 return None
 fp = rdMolDescriptors.GetMorganFingerprintAsBitVect(mol, radius, nBits=n_bits)
 return np.array(fp, dtype=np.uint8)

* **In `backend/main.py`**: python
def smiles_to_fingerprint(smiles: str):
 mol = Chem.MolFromSmiles(smiles)
 if mol is None:
 return None
 fp = rdMolDescriptors.GetMorganFingerprintAsBitVect(
 mol, FINGERPRINT_RADIUS, nBits=FINGERPRINT_BITS
)
 return np.array(fp, dtype=np.uint8).reshape(1, -1)
```
```

5. Handling Invalid Molecules

If a user submits an chemically impossible string like `"InvalidMolecule123"`, RDKit's `Chem.MolFromSmiles()` returns `None`. Our code detects this:

* In `train_model.py`: Invalid rows are safely dropped from training.

* In `backend/main.py`: The API catches `features is None` and immediately raises an **HTTP 422 Unprocessable Entity** exception with a descriptive error message:

```
json $\\frac{ "detail": "Invalid SMILES string 'InvalidMolecule123'. Chemistry parser could not resolve structure." }
```

PART 8 — FEATURE MATRIX X AND TARGET VECTOR y

1. Conceptual and Mathematical Anatomy

In our supervised learning formulation:

* **Feature Matrix X** : A 2D array of dimension $(N \times P)$ where:

* $N = 6,839$ (number of valid molecular samples).

* $P = 2,048$ (number of binary Morgan fingerprint bits per sample).

* $X_{\frac{i}{j}} \in \{0, 1\}$ represents whether molecule i contains the chemical sub-structure corresponding to hashed bit j .

* **Target Vector y** : A 1D array of dimension $(N,)$ where:

* $y_i \in \{0, 1\}$ represents whether molecule i is **inactive** (0) or **active** (1).

2. Matrix Representation Diagram

```

$$X = \begin{bmatrix} x_{\frac{1}{0}} & x_{\frac{1}{1}} & x_{\frac{1}{2}} & \dots & x_{\frac{1}{2047}} \\ x_{\frac{2}{0}} & x_{\frac{2}{1}} & x_{\frac{2}{2}} & \dots & x_{\frac{2}{2047}} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{\frac{6839}{0}} & x_{\frac{6839}{1}} & x_{\frac{6839}{2}} & \dots & x_{\frac{6839}{2047}} \end{bmatrix}$$

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{6839} \end{bmatrix}$$

```

3. Pipeline Data Transitions: Raw Data to Scikit-Learn Fit

```
[Raw CSV File] (6839 rows, 4 columns: molecule_chembl_id, smiles, standard_value, bioactivity_class)
├─ Extract 'smiles' column → RDKit Fingerprint Loop → X (Shape: 6839, 2048; uint8)
├─ Extract 'bioactivity_class' → Map {"active":1, "inactive":0} → y (Shape: 6839,; int64)
└─
    ▼
[Train-Test Split (70/30)]
├─ X_dev (4787, 2048), y_dev (4787,) → Passed to 10-Fold CV & model.fit(X_dev, y_dev)
└─ X_hold (2052, 2048), y_hold (2052,) → Passed to model.predict(X_hold)
```

PART 9 — DATA SPLITTING METHODOLOGY

1. The 70/30 Partition Protocol

To replicate the methodology of the base research paper, the dataset is divided into two distinct partitions:

1. **70% Development Set** ($N = 4,787$ samples):

* Dedicated entirely to internal model training, algorithmic experimentation, and 10-Fold Stratified Cross-Validation.

2. **30% Final Holdout Set** ($N = 2,052$ samples):

* Quarantined from the start. It is **never seen** by any model during feature scaling, cross-validation, or hyperparameter selection. It serves as the final, unbiased test ground.

2. Stratification: Why It Is Crucial

- **Definition of Stratification:** Ensuring that the proportion of target classes in each split mirrors the exact proportion in the full dataset.
- **Why We Use It:** Our dataset has 73.17% active and 26.83% inactive molecules. If a random unstratified split were performed, the holdout set could accidentally receive 35% inactives while the training set received only 22%, causing a distribution mismatch.
- **Code Implementation:**

```
python X_dev, X_hold, y_dev, y_hold = train_test_split( X, y, test_size=0.30, random_state=42,
stratify=y )
```

3. Reproducibility & The Random State

- We fix `random_state=42` across all data splits, cross-validation folds, and stochastic estimators. This ensures that every run produces the exact same partitioning, allowing fair, deterministic comparisons across all 5 candidate models.

PART 10 — 10-FOLD CROSS-VALIDATION

1. What is 10-Fold Stratified Cross-Validation?

- **Definition:** A statistical re-sampling technique where the 70% development dataset is divided into 10 mutually exclusive, equally sized subsets (folds), each maintaining the original 73:27 class ratio.
- **Procedure:** The algorithm runs for 10 sequential iterations. In each iteration $k \in \{1, 2, \dots, 10\}$:
- **9 folds** (90% of development data $\approx 4,308$ samples) are combined to form the training subset.
- **1 fold** (10% of development data ≈ 479 samples) is held out as the validation subset.
- The model is fitted on the 9 folds and evaluated on the 1 validation fold, recording the validation F1-score.
- After 10 iterations, every single sample in the development set has been used for validation exactly once.

2. Cross-Validation Diagram

```
Fold 1 : [ VAL ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ]
Fold 2 : [ TRAIN ] [ VAL ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ]
Fold 3 : [ TRAIN ] [ TRAIN ] [ VAL ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ]
...
Fold 10: [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ TRAIN ] [ VAL ]

Mean 10-Fold CV Score = (F1_1 + F1_2 + ... + F1_10) / 10
Standard Deviation    = sqrt( sum( (F1_k - Mean)^2 ) / 10 )
```

3. Why Cross-Validation Does NOT Mean Saving 10 Models

- A common mentor question: "Did you save 10 different models in your pkl file?"
- **Answer: No.** Cross-validation is strictly a diagnostic tool to assess algorithmic stability and variance. Once the 10-fold CV score is recorded, a **single final model instance** is trained on the **entire 70% development set** (4,787 samples) and evaluated on the 30% holdout set. Only that single final model is serialized to `best_model.pkl`.

4. Windows-Safe Sequential Execution

In our production `train_model.py`, we execute folds sequentially rather than relying on scikit-learn's `n_jobs=-1` inside `cross_val_score`:

```
def run_10fold_cv(model_factory, X_dev, y_dev, skf):
    f1_list = []
    for train_idx, val_idx in skf.split(X_dev, y_dev):
        X_tr, X_val = X_dev[train_idx], X_dev[val_idx]
        y_tr, y_val = y_dev[train_idx], y_dev[val_idx]

        clf = model_factory()
        clf.fit(X_tr, y_tr)
        preds = clf.predict(X_val)
        f1_list.append(f1_score(y_val, preds, zero_division=0))

    return float(np.mean(f1_list)), float(np.std(f1_list))
```

- **Why:** On Windows systems, Python's multiprocessing backend (`joblib` / `loky`) frequently encounters deadlocks when spawning nested worker pools alongside compiled C++ libraries (RDKit and XGBoost). Sequential execution guarantees 100% stability and deterministic results.
-

PART 11 — THE FIVE MACHINE LEARNING ALGORITHMS

Model A: XGBoost (Extreme Gradient Boosting) — WINNER 🏆

- **Core Idea:** An ensemble of shallow decision trees constructed **sequentially**. Each new tree learns to predict the pseudo-residuals (gradients of the loss function) made by all prior trees combined.

- **Mathematical Objective Function:**

$$\mathcal{L} = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \Omega(f)$$

where $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$ is the regularization term penalizing tree complexity and leaf weights.

- **Exact Parameters Used in Our Project:**

```
python xgb.XGBClassifier( n_estimators=150, # Builds 150 sequential boosting rounds max_depth=6, #  
Maximum tree depth of 6 levels learning_rate=0.15, # Shrinkage factor eta = 0.15 to prevent overfitting  
eval_metric="logloss", # Binary cross-entropy optimization random_state=42, verbosity=0 )
```

- **Strengths in This Project:** Excels on sparse 2048-bit representations; built-in L_2 leaf regularization prevents memorization of noise; handles non-linear bit correlations exceptionally well.
- **Decision Boundary:** Highly flexible, piece-wise non-linear decision boundary capable of isolating dense sub-clusters in 2048-dimensional bit space.

Model B: Random Forest Classifier

- **Core Idea:** An ensemble of deep decision trees trained in **parallel** using **Bootstrap Aggregating (Bagging)** and random feature sub-spacing.
 - **How It Works:**
 1. Draws $B = 100$ bootstrap datasets (sampling with replacement) from the training data.
 2. At each node of each tree, randomly samples $\sqrt{P} = \sqrt{2048} \approx 45$ features to find the best split using Gini impurity.
 3. Averages the class probability predictions across all 100 independent trees.
 - **Exact Parameters Used:**
- ```
python RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
```
- **Strengths:** Robust to outliers, low variance, impossible to overfit by simply adding more trees.
  - **Limitations:** Slower inference times than a single gradient-boosted tree; larger serialized memory footprint.

---

### Model C: Linear Support Vector Machine (LinearSVC)

- **Core Idea:** Constructs a maximum-margin separating hyperplane in 2048-dimensional space that maximizes the geometric distance between active and inactive compounds:
$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i \mathbf{w}^T \mathbf{x}_i + b)$$
  - **Exact Pipeline Used:**
- ```
python Pipeline([ ("scaler", StandardScaler(with_mean=False)), ("clf", LinearSVC(C=1.0, max_iter=2000,
```

```
random_state=42)) ])
```

(Note: `with_mean=False` preserves matrix sparsity).

- **Probability Generation:** `LinearSVC` does not have a native `predict_proba()` method. Our code extracts raw signed distances using `decision_function()` and maps them through a standard logistic sigmoid function:
$$P(y=1|\mathbf{x}) = \frac{1}{1 + e^{-\text{score}}}$$
 - **Limitations:** Strictly linear decision boundary; cannot capture synergistic combinations where bit A and bit B must both be present to trigger activity.
-

Model D: Gaussian Naive Bayes (GaussianNB)

- **Core Idea:** A probabilistic classifier applying Bayes' Theorem under the strong ("naive") assumption of conditional feature independence given the class:
$$P(y=c|\mathbf{x}) \propto P(y=c) \prod_{j=1}^{2048} P(x_j|y=c)$$
 - **Exact Configuration:**

```
python GaussianNB()
```
 - **Limitations in This Project:**
 - Assumes Gaussian (bell-curve) continuous likelihoods on binary bit flags (\$0\$ or \$1\$).
 - Assumes chemical fragments are mutually independent, which is chemically false (e.g., if a phenyl ring fragment is present, adjacent aromatic C-H fragments are guaranteed to be present).
 - Consequently achieved the **lowest Holdout F1-score (0.5586)**.
-

Model E: RBF Network (RBFSampler + SGDClassifier)

- **Core Idea:** Approximates an infinite-dimensional Radial Basis Function (Gaussian) kernel in a finite, explicit feature space using **Random Fourier Features** (Rahimi & Recht, 2007).
 - **Exact Pipeline Used:**

```
python Pipeline([ ("scaler", StandardScaler(with_mean=False)), ("rbf_sampler", RBFSampler(gamma=0.01, n_components=500, random_state=42)), ("clf", SGDClassifier(loss="log_loss", max_iter=1000, random_state=42)) ])
```
 - **How It Learns:**
 1. Projects the 2048-bit input into a 500-dimensional Monte Carlo random Fourier space approximating the Gaussian kernel $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$.
 2. Fits a linear logistic regression classifier using Stochastic Gradient Descent on the projected features.
 - **Limitations:** Highly sensitive to the choice of γ ; lower specificity on imbalanced bit vectors.
-

PART 12 — MODEL TRAINING

1. Training Initialization & Execution

When `python train_model.py` is invoked:

1. Memory structures are allocated for the feature array X ($4,787 \times 2,048$) and label array y ($4,787$).
2. For each candidate model, `model.fit(X_dev, y_dev)` is called.
3. During `.fit()`:

- * **XGBoost**: Calculates first-order gradients g_i and second-order Hessians h_i of the logistic loss, iteratively building 150 regression trees that greedily split features to maximize gain.

- * **Random Forest**: Builds 100 decision trees via bootstrap samples and random 45-feature subsets.

- * **Linear SVM**: Solves the coordinate descent quadratic optimization problem to find optimal weights $\mathbf{w} \in \mathbb{R}^{\frac{1}{2048}}$.

2. Fair Comparison Protocol

All 5 candidate models receive the **exact same feature matrices** (X_{dev} , X_{hold}) and **exact same labels** (y_{dev} , y_{hold}). No model receives privileged feature engineering or altered data splits.

PART 13 — MODEL EVALUATION

1. The Confusion Matrix

A 2×2 contingency table contrasting true laboratory bioactivity against model predictions:

| | Model Predicted Inactive (0) | Model Predicted Active (1) |
|---------------------|------------------------------|----------------------------|
| Actual Inactive (0) | True Negative (TN) | False Positive (FP) |
| Actual Active (1) | False Negative (FN) | True Positive (TP) |

- **True Positive (TP)**: Molecule is genuinely active, and model correctly predicted `active`.
- **True Negative (TN)**: Molecule is genuinely inactive, and model correctly predicted `inactive`.
- **False Positive (FP)** (Type I Error): Molecule is inactive, but model incorrectly predicted `active`.
- *Drug Discovery Impact*: Wastes lab budget synthesizing a dead molecule.
- **False Negative (FN)** (Type II Error): Molecule is active, but model incorrectly predicted `inactive`.
- *Drug Discovery Impact*: Misses a potential multi-million-dollar blockbuster drug.

2. Comprehensive Metric Formulations

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

* *Meaning*: Overall percentage of correct predictions.

* *Why Insufficient Alone*: With 73.17% active molecules, predicting 100% active yields 73.17% accuracy while completely failing to identify inactives.

Precision (Positive Predictive Value)

$$\text{Precision} = \frac{TP}{TP + FP}$$

* *Meaning*: When the model flags a compound as `active`, how often is it truly active?

* *Drug Discovery Impact*: Directly measures wet-lab cost efficiency.

Recall / Sensitivity (True Positive Rate)

$$\text{Recall} = \frac{TP}{TP + FN}$$

* *Meaning*: Out of all genuinely active COX-2 inhibitors in the world, what percentage did the model successfully find?

Specificity (True Negative Rate)

$$\text{Specificity} = \frac{TN}{TN + FP}$$

* *Meaning*: Out of all inactive compounds, what percentage did the model correctly discard?

F1-Score (Harmonic Mean)

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

* *Meaning*: Balances precision and recall into a single score. Highly resilient to class imbalance.

ROC-AUC (Receiver Operating Characteristic - Area Under Curve)

- Evaluates the trade-off between True Positive Rate ($TP / [TP + FN]$) and False Positive Rate ($FP / [TN + FP]$) across every conceivable decision threshold from 0.0 to 1.0.
- *Interpretation:* The probability (0.0 to 1.0) that the model will assign a higher active probability to a randomly chosen active molecule than to a randomly chosen inactive molecule. 0.5 represents random guessing; 1.0 represents a flawless classifier.

PART 14 — BASE PAPER AUC VS. MODERN ROC-AUC

1. Two Distinct Mathematical Definitions

In cheminformatics literature, the acronym "AUC" can refer to two completely different mathematical entities:

1. Base Paper AUC (Balanced Accuracy):

$$\text{AUC}_{\text{paper}} = \frac{\text{Sensitivity} + \text{Specificity}}{2} = \frac{\frac{\text{TP}}{\text{TP} + \text{FN}} + \frac{\text{TN}}{\text{TN} + \text{FP}}}{2}$$

* Calculated from the **binary confusion matrix at a single fixed decision threshold** (default 0.5).

* Measures the unweighted arithmetic mean of accuracy across both classes.

2. Modern ROC-AUC (Area Under the ROC Curve):

$$\text{ROC-AUC} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(t)) \, dt$$

* Calculated by sweeping the decision threshold $\theta \in [0.0, 1.0]$ across continuous predicted probabilities, plotting Sensitivity against $(1 - \text{Specificity})$, and integrating the area via the trapezoidal rule.

2. How Our Project Handles Both

- In `train_model.py` and `backend/main.py`:
 - We report **Sensitivity (Recall)** and **Specificity** directly, allowing anyone to compute the base paper's metric instantly: $\text{Paper AUC} = (0.9407 + 0.5118) / 2 = 0.7263$.
 - We compute and report modern **ROC-AUC** (`0.8677`) via scikit-learn's `roc_auc_score(y_hold, y_proba)`.
-

PART 15 — CONFUSION MATRICES ACROSS ALL 5 MODELS

On the untouched 30% Holdout Set (\$N = 2,052\$ molecules: \$1,502\$ Actives, \$550\$ Inactives):

1. XGBoost (Winner)

- $\text{\text{\frac{TP}}{}} = 1,413 \quad | \quad \text{\text{\frac{FN}}{}} = 89$ (Recall = 94.07%)
- $\text{\text{\frac{TN}}{}} = 281 \quad | \quad \text{\text{\frac{FP}}{}} = 269$ (Specificity = 51.18%)
- *Analysis:* Extremely high hit-rate on active molecules while successfully filtering more than half of all inactives.

2. Random Forest

- $\text{\text{\frac{TP}}{}} = 1,394 \quad | \quad \text{\text{\frac{FN}}{}} = 108$ (Recall = 92.80%)
- $\text{\text{\frac{TN}}{}} = 296 \quad | \quad \text{\text{\frac{FP}}{}} = 254$ (Specificity = 53.90%)

3. Linear SVM

- $\text{\text{\frac{TP}}{}} = 1,275 \quad | \quad \text{\text{\frac{FN}}{}} = 227$ (Recall = 84.88%)
- $\text{\text{\frac{TN}}{}} = 334 \quad | \quad \text{\text{\frac{FP}}{}} = 216$ (Specificity = 60.80%)

4. Naive Bayes

- $\text{\text{\frac{TP}}{}} = 615 \quad | \quad \text{\text{\frac{FN}}{}} = 887$ (Recall = 40.97%)
- $\text{\text{\frac{TN}}{}} = 464 \quad | \quad \text{\text{\frac{FP}}{}} = 86$ (Specificity = 84.39%)
- *Analysis:* Severely underpredicts the active class due to violated independence assumptions.

5. RBF Network

- $\text{\text{\frac{TP}}{}} = 1,415 \quad | \quad \text{\text{\frac{FN}}{}} = 87$ (Recall = 94.20%)
- $\text{\text{\frac{TN}}{}} = 56 \quad | \quad \text{\text{\frac{FP}}{}} = 494$ (Specificity = 10.16%)
- *Analysis:* Collapsed towards predicting mostly active, resulting in an unacceptably low specificity of 10.16%.

PART 16 — FINAL MODEL COMPARISON & SELECTION

1. Official Experimental Results Table (30% Holdout Set)

| Model Architecture | 10-Fold CV Mean F1 | Holdout Accuracy | Precision | Recall (Sens.) | Specificity | Holdout F1-Score | Modern ROC-AUC | Selection Status |
|--------------------|--------------------|------------------|-----------|----------------|-------------|------------------|----------------|------------------------|
| XGBoost | 0.8853 | 82.55% | 0.8400 | 0.9407 | 0.5118 | 0.8875 | 0.8677 | 🏆 Auto-Selected Winner |
| Random Forest | 0.8936 | 82.36% | 0.8458 | 0.9280 | 0.5390 | 0.8850 | 0.8620 | Baseline Runner-Up |
| Linear SVM | 0.8585 | 78.41% | 0.8550 | 0.8488 | 0.6080 | 0.8519 | 0.7867 | Linear Baseline |
| RBF Network | 0.8168 | 71.64% | 0.7407 | 0.9420 | 0.1016 | 0.8293 | 0.5975 | Neural Baseline |
| Naive Bayes | 0.5221 | 52.63% | 0.8773 | 0.4097 | 0.8439 | 0.5586 | 0.7401 | Probabilistic Baseline |

2. Automatic Selection Logic

In `train_model.py`:

```
sorted_df = res_df.sort_values(
    by=["Holdout F1", "ROC-AUC"],
    ascending=[False, False]
).reset_index(drop=True)

best_row = sorted_df.iloc[0]
best_name = best_row["Model"] # Evaluates to "XGBoost"
```

- **Primary Selection Criterion:** Highest Holdout F1-Score (`0.8875`).
- **Tie-Breaker Criterion:** Highest Holdout ROC-AUC (`0.8677`).
- **Why Programmatic Selection Matters:** We do not hardcode or force XGBoost. The script sorts the benchmark table purely on merit, proving objective scientific methodology.

PART 17 — MODEL SERIALIZATION

1. What is Serialization?

- **Definition:** The process of translating an in-memory Python object hierarchy (the trained classifier, its learned tree structures, split thresholds, and leaf weights) into a persistent byte stream that can be stored on disk and restored later.
- **Why We Serialize:** Training takes several minutes and requires the dataset. Inference takes 5 milliseconds and requires only the model object. By saving the model, the production web application can start up and serve predictions instantly without retraining.

2. best_model.pkl

- **Library Used:** `joblib` (`joblib.dump()` and `joblib.load()`).
- **Location:** `backend/model/best_model.pkl` (File size: ~15 MB).
- **What is Inside:** The complete trained C++ and Python state of the winning `XGBClassifier` instance, including all 150 gradient-boosted trees and their split indices across the 2048 fingerprint dimensions.

3. metadata.json

- **Location:** `backend/model/metadata.json`
- **Purpose:** Provides a decoupled, lightweight configuration contract between the training script and the backend API.
- **Complete Field Breakdown:**
 - `best_model_name` : `"XGBoost"` (Identifies which architecture won).
 - `label_mapping` : `{ "active": 1, "inactive": 0 }` (Target encoding).
 - `reverse_mapping` : `{ "1": "active", "0": "inactive" }` (Decodes integer back to string).
 - `fingerprint_radius` : `2` (Guarantees backend uses radius 2).
 - `fingerprint_bits` : `2048` (Guarantees backend uses 2048 bits).
 - `best_metrics` : Full dictionary of winning metrics (Holdout F1: `0.8875` , AUC: `0.8677`).
 - `all_results` : Array of metric objects across all 5 benchmarked models.

PART 18 — BACKEND ARCHITECTURE (FastAPI)

1. Why FastAPI?

- **High Performance:** Built on Starlette and Pydantic, running on the asynchronous Uvicorn ASGI server, rivaling NodeJS and Go in throughput.
- **Automatic Interactive Documentation:** Generates interactive Swagger UI docs automatically at `/docs` and ReDoc at `/redoc`.
- **Native Type Validation:** Automatically validates incoming JSON request schemas using Python type hints.

2. Startup Lifespan & Path Resolution

In `backend/main.py`:

```
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
PROJECT_ROOT = os.path.dirname(CURRENT_DIR)
MODEL_PATH = os.path.join(CURRENT_DIR, "model", "best_model.pkl")
METADATA_PATH = os.path.join(CURRENT_DIR, "model", "metadata.json")

best_model = joblib.load(MODEL_PATH)
with open(METADATA_PATH, "r", encoding="utf-8") as f:
    metadata = json.load(f)
```

- **Robustness:** Using `os.path.abspath(__file__)` guarantees that the server boots cleanly whether launched from the project root, from the `backend/` sub-directory, or via automated process monitors.

3. API Endpoints

Endpoint 1: `GET /`

- **Purpose:** Root welcome route and discovery endpoint.
- **Response:**

```
json $\frac{ "message": "Bioactive Molecule Predictor API", "model": "XGBoost", "docs": "/docs", "app": "/app/" }
```

Endpoint 2: `GET /health`

- **Purpose:** Health check for automated monitoring and frontend heartbeat.
- **Response:**

```
json $\frac{ "status": "healthy", "model_loaded": "XGBoost" }
```

Endpoint 3: `POST /predict`

- **Purpose:** The primary machine learning inference route.
 - **Request Body:**
- ```
json $\frac{ "smiles": "CC(=O)Oc1ccccc1C(=O)O" }
```
- **Processing Steps:**

1. Validates `smiles` string is non-empty.
2. Calls `smiles_to_fingerprint(smiles)`.
3. Raises HTTP 422 if RDKit returns `None`.

4. Calls `best_model.predict(features)` .
5. Calls `best_model.predict_proba(features)` to retrieve class probability.
6. Maps integer prediction via `REVERSE_MAPPING` .

- **Success Response (HTTP 200):**

```
json $\frac{ \{ "prediction": "inactive", "probability": 0.6605, "model": "XGBoost", "smiles":
"CC(=O)Oc1ccccc1C(=O)O" \}
```

**Static Files Mount:** `/app`

- Mounts the `frontend/` directory to serve `index.html` , `style.css` , and `script.js` directly over HTTP at `http://127.0.0.1:8000/app/` .

## PART 19 — PYDANTIC REQUEST VALIDATION

---

### 1. What is Pydantic?

- **Definition:** A data validation and parsing library for Python that enforces type hints at runtime, providing user-friendly errors when data is invalid.

- In `backend/main.py` :

```
```python
from pydantic import BaseModel

class PredictRequest(BaseModel):
    smiles: str
```
```

### 2. How Pydantic Protects the Machine Learning Pipeline

- If a client submits malformed JSON (e.g.,  `$\frac{\text{"wrong\_key": 123}}$`  or a number instead of a string), FastAPI intercepts the request before it reaches our chemistry code and automatically returns an **HTTP 422 Unprocessable Entity** response specifying exactly which field failed validation.
-

## PART 20 — API AND HTTP NETWORKING

---

### 1. Core HTTP Concepts

- **REST (Representational State Transfer):** An architectural style for networked systems relying on stateless client-server communication using standard HTTP methods.
  - **HTTP GET vs. POST:**
  - `GET` : Used for retrieving data without altering server state ( `/health` , `/` ).
  - `POST` : Used for sending data payloads to the server for processing ( `/predict` ). We use `POST` because molecular SMILES strings can be long and contain special characters (like `#` , `=` , `/` , `\` ) that would be corrupted if placed in a GET URL query string.
  - **JSON (JavaScript Object Notation):** The universal text-based data interchange format used between our JavaScript client and FastAPI.
  - **HTTP Status Codes:**
  - `200 OK` : Request succeeded.
  - `422 Unprocessable Entity` : Input failed validation (empty or invalid SMILES).
  - `500 Internal Server Error` : Unhandled server exception during inference.
  - **CORS (Cross-Origin Resource Sharing):**
  - When opening `frontend/index.html` via `file:///` or an external domain, the browser's Same-Origin Policy blocks API calls unless the backend explicitly permits it.
  - In `backend/main.py` , `CORSMiddleware` is configured with `allow_origins=["*"]` , enabling smooth testing across all environments.
-

## PART 21 — FRONTEND ARCHITECTURE

---

### 1. Semantic HTML Structure ( `index.html` )

- **Header / Navbar:** Logo, project branding, target enzyme tag, and backend health status indicator badge.
- **Hero / About Section:** Explicit **Problem Statement** and **What This Project Does** cards explaining the drug discovery context.
- **Benchmark Results Section:** A clean, formatted HTML table displaying the holdout comparison results for all 5 models, highlighting XGBoost with a winner badge.
- **Prediction Workspace (Grid Layout):**
  - *Left Column (Input Panel):* Textarea for SMILES input, "Load Example" cycle button, "Clear" button, inline error message container, and "Predict Bioactivity" submit button.
  - *Right Column (Results Panel):* Dynamic display area showing empty state placeholder, animated loading spinner, and the final prediction badge.

### 2. CSS Styling & Layout ( `style.css` )

- Built with modern CSS custom properties (variables) for consistent colors ( `--primary: #2563eb` , `--success: #16a34a` , `--error: #dc2626` ).
- Fully responsive layout utilizing CSS Flexbox and CSS Grid, seamlessly adapting between 4K monitors, laptops, and mobile screens.

### 3. JavaScript Client ( `script.js` )

- Written in modern, modular Vanilla JavaScript (no heavy frameworks like React or Vue required).
- **Key Functions:**
  - `checkBackendHealth()` : Queries `GET /health` with an `AbortSignal.timeout(3000)` to update the navbar status badge ( `Online` vs `Offline` ).
  - `loadExample()` : Cycles through pre-defined drug SMILES (Ethanol, Aspirin, Ibuprofen, Celecoxib) into the input box.
  - `validateInput()` : Checks for empty strings, whitespace, or pure digits before dispatching network requests.
  - `predictMolecule()` : The asynchronous orchestrator managing loading spinners, dispatching `fetch()` , and passing data to the renderer.
  - `displayPrediction(data)` : Constructs HTML cards with active/inactive badges and confidence percentages.

## PART 22 — THE FETCH API & CLIENT-SERVER COMMUNICATION

---

### 1. Anatomical Breakdown of the Fetch Call

In `frontend/script.js`:

```
const response = await fetch(`${API_BASE}/predict`, {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ smiles: smiles })
});
const data = await response.json();
```

- ``${API_BASE}/predict``: Dynamically resolves to `http://127.0.0.1:8000/predict` (or current origin when served via `/app/`).
  - `method: "POST"`: Specifies HTTP POST verb.
  - `headers: { "Content-Type": "application/json" }`: Informs FastAPI that the payload is formatted as JSON.
  - `body: JSON.stringify(...)`: Serializes the JavaScript object `{ smiles: "CCO" }` into the raw string `'{"smiles": "CCO"}'`.
  - `await response.json()`: Asynchronously parses the incoming JSON response body into a native JavaScript object.
-

## PART 23 — COMPLETE END-TO-END FLOW (22 Granular Steps)

When a user enters a SMILES string and clicks "Predict Bioactivity":

- Step 1 (User Action):** The user types or pastes a SMILES string (e.g., Celecoxib: `Cc1ccc(-c2cc(C(F)(F)F)nn2-c2ccc(S(N)(=O)=O)cc2)cc1`) into the textarea.
- Step 2 (DOM Storage):** The browser engine stores the text in the HTML textarea node (`#smilesInput`).
- Step 3 (Event Trigger):** The user clicks the button, triggering the click event listener registered in `script.js`.
- Step 4 (Frontend Validation):** `validateInput()` verifies that the string is non-empty and contains no illegal spaces.
- Step 5 (UI State Update):** `setLoadingState(true)` disables the button, displays the animated spinner, and hides previous results.
- Step 6 (JSON Serialization):** `JSON.stringify($\frac{ smiles: smiles })` packages the payload into a standard JSON string.
- Step 7 (HTTP Transmission):** The browser's networking stack transmits an HTTP POST packet over local loopback (`127.0.0.1:8000`).
- Step 8 (FastAPI Interception):** The Uvicorn ASGI server receives the raw TCP stream and hands the request to FastAPI.
- Step 9 (Pydantic Schema Validation):** Pydantic parses the JSON body into an instance of `PredictRequest`, ensuring the `smiles` string attribute exists.
- Step 10 (Backend Feature Extraction):** FastAPI calls `smiles_to_fingerprint(clean_smiles)`.
- Step 11 (RDKit Parsing):** RDKit's C++ parser converts the SMILES string into an in-memory 2D molecular graph (`Chem.Mol`).
- Step 12 (Morgan Fingerprint Calculation):** RDKit traverses atomic neighborhoods to radius 2, hashes fragments, and constructs a 2048-bit `ExplicitBitVect`.
- Step 13 (NumPy Reshaping):** The bit vector is cast into a `uint8` NumPy array and reshaped to `(1, 2048)`.
- Step 14 (Model Invocation):** The serialized `best_model` (XGBoost) executes `predict(features)`, outputting integer `1` (Active).
- Step 15 (Probability Calculation):** `predict_proba(features)` computes class probabilities `[P(inactive)=0.0274, P(active)=0.9726]`.
- Step 16 (Label Decoding):** The integer `1` is mapped back to the human-readable string `"active"` via `REVERSE_MAPPING`.
- Step 17 (JSON Payload Construction):** FastAPI builds a dictionary:  

```
json $\frac{ "prediction": "active", "probability": 0.9726, "model": "XGBoost", "smiles": "..."}$
```
- Step 18 (HTTP Response):** The server sends an HTTP 200 OK response with the JSON body back to the client.
- Step 19 (Client Reception):** `await response.json()` deserializes the HTTP response body into a JavaScript object.
- Step 20 (Loading Reset):** `setLoadingState(false)` restores the button and dismisses the spinner.
- Step 21 (DOM Rendering):** `displayPrediction(data)` injects HTML markup: a green "Active" badge, "97.3% Confidence", and model metadata.
- Step 22 (User Comprehension):** The user observes the completed, color-coded prediction on their screen.

# PART 24 — TRAINING TIME VS. PREDICTION TIME

A foundational concept for any technical defense is the strict division between training and inference:

| Dimension      | Training Phase ( <code>train_model.py</code> )                           | Prediction Phase ( <code>backend/main.py</code> )                           |
|----------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Trigger        | Executed once by the developer during model development.                 | Executed in real-time whenever a user submits a SMILES string.              |
| Input Data     | Full dataset: <code>CHEMBL230_Preprocessed_Data.csv</code> (6,839 rows). | A single molecular SMILES string from the user.                             |
| Execution Time | Several minutes (10-fold CV across 5 models).                            | 5 to 50 milliseconds.                                                       |
| Hardware Use   | High CPU utilization across all cores.                                   | Lightweight single-thread execution.                                        |
| Output         | <code>best_model.pkl</code> and <code>metadata.json</code> .             | Ephemeral JSON response sent over HTTP.                                     |
| Algorithms     | Model fitting, cross-validation, loss gradient descent.                  | Model evaluation ( <code>predict()</code> , <code>predict_proba()</code> ). |

## PART 25 — FILE-TO-FILE CONNECTION MAP



### Detailed Data Flow Between Files:

- `CHEMBL230_Preprocessed_Data.csv`  $\rightarrow$  `train_model.py` : Supplies the raw tabular data (SMILES and class labels) for model fitting.
- `train_model.py`  $\rightarrow$  `backend/model/best_model.pkl` : Exports the serialized C++ XGBoost decision trees.
- `train_model.py`  $\rightarrow$  `backend/model/metadata.json` : Exports the operational configuration (model name, bit counts, label mapping, evaluation metrics).
- `backend/model/*`  $\rightarrow$  `backend/main.py` : Read once during FastAPI startup to populate memory.
- `frontend/index.html`  $\rightarrow$  `frontend/script.js` : Provides DOM node references for event listening.
- `frontend/script.js`  $\leftrightarrow$  `backend/main.py` : Bridges the browser and server over HTTP using JSON payloads.

## PART 26 — TECHNOLOGY STACK

---

Every technology in this project was selected for a specific technical reason:

1. **Python 3.10+**: The premier programming language for data science, offering robust C-extensions for scientific computing.
  2. **Pandas**: High-performance tabular data structures ( `DataFrame` ) used to load and parse the ChEMBL CSV dataset.
  3. **NumPy**: Low-level contiguous memory arrays used to represent the 2D feature matrix `XX` and perform high-speed vector math.
  4. **RDKit**: Open-source cheminformatics library used to parse chemical SMILES into molecular graphs and generate 2048-bit Morgan fingerprints.
  5. **Scikit-Learn**: Industry-standard ML toolkit providing stratified data splitting ( `train_test_split` ), cross-validation ( `StratifiedKFold` ), baseline models (Random Forest, SVM, Naive Bayes), and evaluation metrics ( `f1_score` , `roc_auc_score` ).
  6. **XGBoost**: High-performance gradient boosting library implemented in optimized C++ with exact greedy tree pruning and regularization.
  7. **Joblib**: Optimized disk serialization for large NumPy arrays and machine learning estimators.
  8. **FastAPI**: Modern, asynchronous web framework for building Python REST APIs with automatic OpenAPI/Swagger documentation.
  9. **Pydantic**: Data validation library enforcing strict typing on API request bodies.
  10. **Uvicorn**: Lightning-fast ASGI (Asynchronous Server Gateway Interface) web server implementation for Python.
  11. **HTML5**: Semantic web markup structuring the client user interface.
  12. **CSS3**: Stylesheet language utilizing modern CSS Variables, Flexbox, and CSS Grid for a fully responsive user interface.
  13. **Vanilla JavaScript (ES6+)**: Native browser scripting utilizing asynchronous `fetch()` , DOM manipulation, and promises without the bloat of external frameworks.
-

## PART 27 — ERROR HANDLING AND DEFENSIVE PROGRAMMING

Our application is built defensively across all architectural tiers:

| Failure Mode                                 | Layer                                  | System Response                                                                                                                      |
|----------------------------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Empty SMILES string</b>                   | Frontend<br>( <code>script.js</code> ) | Client-side validation halts submission; displays <i>"Please enter a SMILES string before predicting."</i>                           |
| <b>Malformed SMILES (e.g. spaces/digits)</b> | Frontend<br>( <code>script.js</code> ) | Regex validation flags invalid format before network call is made.                                                                   |
| <b>Chemically invalid SMILES ( XYZ123 )</b>  | Backend<br>( <code>main.py</code> )    | RDKit returns <code>None</code> ; API catches it and returns <b>HTTP 422</b> with descriptive JSON.                                  |
| <b>Empty string reaches backend</b>          | Backend<br>( <code>main.py</code> )    | Handled via <code>if not clean_smiles: raise HTTPException(status_code=422)</code> .                                                 |
| <b>Model file missing ( best_model.pkl )</b> | Backend<br>( <code>main.py</code> )    | App startup crashes immediately with <b>RuntimeError</b> , alerting admin to run <code>train_model.py</code> .                       |
| <b>FastAPI backend offline</b>               | Frontend<br>( <code>script.js</code> ) | <code>fetch()</code> promise rejects; <code>catch</code> block catches error and renders clear offline troubleshooting instructions. |
| <b>Network timeout</b>                       | Frontend<br>( <code>script.js</code> ) | Health check utilizes <code>AbortSignal.timeout(3000)</code> to prevent the browser from hanging indefinitely.                       |

## PART 28 — SECURITY AND VALIDATION FUNDAMENTALS

---

- **No Arbitrary Code Execution:** The API strictly accepts string input representing SMILES notation. It never executes `eval()` or passes user input to shell commands.
- **Input Sanitization:** SMILES strings are stripped of leading/trailing whitespace and validated against chemical parsers.
- **Strict CORS Policy:** While currently set to `allow_origins=["*"]` for frictionless local development, the middleware can be restricted to specific production domains with a single line of code.
- **Stateless REST Architecture:** The backend stores no session state, preventing session hijacking or cross-site request forgery (CSRF) vulnerabilities.

## PART 29 — COMPREHENSIVE MENTOR VIVA QUESTIONS & ANSWERS (100 Q&As)

---

### Category A: Project Basics & Architecture

**Q1: What is the high-level goal of this project?**

**A:** To build a complete machine learning system that predicts whether a chemical compound will inhibit the human Cyclooxygenase-2 (COX-2) enzyme based on its molecular structure (SMILES).

**Q2: What are the primary inputs and outputs of the deployed system?**

**A:** The input is a textual molecular SMILES string. The output is a binary classification ( `active` or `inactive` ), a confidence score (percentage), and the active model name.

**Q3: Why is this problem framed as classification rather than regression?**

**A:** In drug discovery virtual screening, researchers need binary go/no-go decisions at a standard pharmaceutical potency cutoff ( $10^{-6}$  M). Regression on noisy cross-laboratory  $IC_{50}$  values introduces false precision and model instability.

**Q4: Is this supervised or unsupervised learning?**

**A:** Supervised learning, because we train our models on labeled historical assay data where the ground-truth bioactivity class is known.

**Q5: Does the frontend perform any machine learning?**

**A:** No. The frontend handles only UI rendering, user input validation, and asynchronous HTTP communication. All ML inference is executed on the Python backend.

---

### Category B: Drug Discovery & Biology

**Q6: What biological target does this project focus on?**

**A:** Human Cyclooxygenase-2 (COX-2), an enzyme responsible for producing pro-inflammatory prostaglandins.

**Q7: What is the medical significance of inhibiting COX-2?**

**A:** Inhibiting COX-2 relieves pain, reduces inflammation, and lowers fever. Selective COX-2 inhibitors (such as Celecoxib) were designed to provide pain relief without causing the severe gastric ulcers associated with non-selective COX-1 inhibition.

**Q8: What is the difference between COX-1 and COX-2?**

**A:** COX-1 is a constitutive enzyme expressed in most tissues to protect gastric mucosa and regulate platelets. COX-2 is an inducible enzyme expressed primarily at sites of inflammation.

**Q9: What does  $IC_{50}$  mean in simple terms?**

**A:** The concentration of a drug required to reduce enzyme activity by 50%. A lower  $IC_{50}$  indicates a more potent drug.

**Q10: Why is 10,000 nM chosen as the cutoff threshold?**

**A:**  $10,000 \text{ nM} = 10^{-6} \text{ M}$ , the standard pharmaceutical high-throughput screening threshold for declaring biological activity in primary assays.

**Q11: If a molecule is predicted Active, is it guaranteed to be a successful drug?**

**A:** Absolutely not. It only suggests potential enzyme inhibition. It says nothing about toxicity, bioavailability, metabolic stability, or human efficacy.

**Q12: Why is Ethanol predicted as Inactive by our model?**

**A:** Because ethanol does not inhibit the COX-2 enzyme at sub- $10^{-10}$  concentrations. While ethanol is biologically active in the central nervous system, it is inactive against COX-2.

---

### Category C: Dataset & Data Quality

**Q13: Where did the dataset come from?**

**A:** The ChEMBL database (EMBL-EBI), target ID `CHEMBL230`.

**Q14: How many samples and features are in the raw CSV?**

**A:** Exactly 6,839 rows and 4 columns (`molecule_chembl_id`, `smiles`, `standard_value`, `bioactivity_class`).

**Q15: What is the class distribution in the dataset?**

**A:** 5,004 Actives (73.17%) and 1,835 Inactives (26.83%), representing an imbalance ratio of ~2.73:1.

**Q16: Are there missing values in the dataset?**

**A:** No, there are 0 missing values across all columns.

**Q17: Why are there duplicate SMILES rows in the CSV?**

**A:** They correspond to independent bioassays published by different research teams across scientific literature cataloged by ChEMBL.

**Q18: What is target leakage and how did you prevent it?**

**A:** Target leakage occurs when the target or its direct source is used as an input feature. We strictly exclude `standard_value` from the feature matrix  $X$  because it was used to create the `bioactivity_class` label.

---

### Category D: Chemistry & SMILES

**Q19: What does SMILES stand for?**

**A:** Simplified Molecular Input Line Entry System.

**Q20: How does SMILES represent branches and rings?**

**A:** Branches are enclosed in parentheses `( )`; rings are indicated by matching integer pairs following the bonded atoms (e.g., `c1ccccc1`).

**Q21: Can ML models take raw SMILES text as direct inputs?**

**A:** No. SMILES strings have variable lengths and no explicit numerical metrics; ML models require fixed-dimensional numerical feature vectors.

**Q22: What is the SMILES string for Aspirin?**

**A:** `CC(=O)Oc1ccccc1C(=O)O`.

**Q23: What happens if an invalid SMILES string is provided to the system?**

**A:** RDKit fails to parse it and returns `None`. The backend catches this and returns an HTTP 422 error.

---

### Category E: RDKit & Feature Generation

**Q24: What is RDKit?**

**A:** An open-source cheminformatics and chemical modeling library with a C++ core and Python bindings.

**Q25: What exact fingerprint algorithm is used in this project?**

**A:** Morgan Circular Fingerprint (radius  $r=2$ , length 2048 bits), equivalent to ECFP4.

**Q26: What does radius=2 mean?**

**A:** The circular algorithm inspects atomic environments up to 2 chemical bonds away from each central atom, capturing fragments spanning up to 4 bonds in diameter.

**Q27: What is the shape of the feature matrix X?**

**A:** (6839, 2048) with data type uint8.

**Q28: Why do we use bit vectors instead of count vectors?**

**A:** Standard ECFP4 bit vectors capture the presence/absence of functional groups effectively while maintaining high computational efficiency and sparse representation.

**Q29: How do you guarantee feature parity between training and inference?**

**A:** Both `train_model.py` and `backend/main.py` invoke identical RDKit parameters: `radius=2` and `nBits=2048`.

---

## Category F: Data Splitting & Cross-Validation

**Q30: What data split was used?**

**A:** 70% Development Set (4,787 samples) and 30% Untouched Holdout Test Set (2,052 samples).

**Q31: What is stratified splitting?**

**A:** A splitting method that preserves the exact 73:27 ratio of active to inactive classes across both partitions.

**Q32: Why didn't you use 100% of the data for cross-validation?**

**A:** To ensure a true final evaluation, 30% of the data must remain completely untouched during CV to serve as an independent test set.

**Q33: What is 10-Fold Stratified Cross-Validation?**

**A:** Dividing the 70% development set into 10 folds, iteratively training on 9 folds and validating on 1 fold, repeated 10 times so every sample is validated once.

**Q34: Why was cross-validation run sequentially in Python?**

**A:** To avoid multiprocessing worker deadlocks common on Windows platforms when combining scikit-learn with C++ libraries (RDKit, XGBoost).

**Q35: Does 10-fold CV create 10 production models?**

**A:** No. CV is used solely to assess model stability. The final production model is trained once on the full 70% development set.

---

## Category G: Machine Learning Algorithms

**Q36: Which five algorithms were evaluated?**

**A:** XGBoost, Random Forest, Linear SVM, Naive Bayes, and RBF Network.

**Q37: Why did XGBoost achieve the best performance?**

**A:** Its sequential gradient boosting corrects residual errors from previous trees, while built-in  $L_1$  and  $L_2$  regularization prevents overfitting to dense chemical clusters.

**Q38: What hyperparameters were configured for XGBoost?**

**A:** `n_estimators=150`, `max_depth=6`, `learning_rate=0.15`, `eval_metric="logloss"`, and `random_state=42`.

**Q39: How does Random Forest differ from XGBoost?**

**A:** Random Forest trains deep trees independently in parallel via bagging. XGBoost trains shallow trees sequentially via

gradient boosting.

**Q40: Why did Naive Bayes perform poorly?**

**A:** It assumes conditional independence between features, which fails in chemistry where molecular fragments are highly correlated.

**Q41: How did the RBF Network approximate non-linear kernels?**

**A:** Using scikit-learn's `RBFSampler` (Random Fourier Features) with 500 components, followed by an `SGDClassifier` with logistic loss.

**Q42: How does LinearSVC generate probabilities if it lacks predict\_proba?**

**A:** Our code computes raw distance scores using `decision_function()` and maps them through a logistic sigmoid function:  $1 / (1 + e^{-\text{score}})$ .

---

## Category H: Evaluation Metrics & Results

**Q43: What were the final holdout metrics for the winning XGBoost model?**

**A:** Holdout F1-Score: **0.8875**, Accuracy: **82.55%**, ROC-AUC: **0.8677**, Precision: **0.8400**, Recall: **0.9407**, Specificity: **0.5118**.

**Q44: What was the primary model selection metric?**

**A:** Highest Holdout F1-Score, with Holdout ROC-AUC as the tie-breaker.

**Q45: Why is Accuracy misleading on this dataset?**

**A:** With 73.17% active samples, a trivial model predicting everything as active achieves 73.17% accuracy while having 0% specificity.

**Q46: What is the formula for F1-Score?**

**A:**  $F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ .

**Q47: What does Specificity measure in our project?**

**A:** The percentage of genuinely inactive molecules correctly identified as inactive by the model ( $TN / [TN + FP]$ ).

**Q48: What is the difference between base paper AUC and modern ROC-AUC?**

**A:** The base paper defined  $\text{AUC} = (\text{Sensitivity} + \text{Specificity}) / 2$  at a single cutoff. Modern ROC-AUC integrates the area under the True Positive vs False Positive curve across all thresholds.

---

## Category I: Model Serialization & Persistence

**Q49: Which file stores the trained model?**

**A:** `backend/model/best_model.pkl`.

**Q50: Why is Joblib preferred over standard Pickle for machine learning?**

**A:** Joblib is optimized for storing large NumPy arrays and complex scikit-learn/XGBoost structures efficiently.

**Q51: What is the purpose of metadata.json?**

**A:** It stores pipeline configuration (model name, bit size, radius, label mappings, and holdout benchmark results) for decoupled backend use.

---

## Category J: Backend & FastAPI

**Q52: What web framework is used for the backend?**

**A:** FastAPI running on the Uvicorn ASGI server.

**Q53: How does the backend load the model?**

**A:** It loads `best_model.pkl` and `metadata.json` into memory once during application startup.

**Q54: What endpoints are exposed by the API?**

**A:** `GET /` (welcome), `GET /health` (status), `POST /predict` (inference), and `GET /app` (frontend static mount).

**Q55: What HTTP status code is returned for an invalid SMILES?**

**A:** HTTP 422 Unprocessable Entity.

**Q56: What does the Pydantic PredictRequest schema do?**

**A:** It validates that the incoming JSON payload contains a valid `smiles` string before passing it to RDKit.

**Q57: Why is CORS middleware necessary?**

**A:** To allow web browsers loading the frontend from `file://` or other ports to communicate with `http://127.0.0.1:8000`.

---

## Category K: Frontend & Deployment

**Q58: What technologies were used for the frontend?**

**A:** Semantic HTML5, CSS3 with Flexbox/Grid, and Vanilla JavaScript using the Fetch API.

**Q59: How does the frontend handle loading states?**

**A:** It disables the Predict button, displays an animated loading spinner, and hides previous results until the request finishes.

**Q60: How does script.js determine the API URL?**

**A:** If loaded via `file://`, it defaults to `http://127.0.0.1:8000`. If served via the backend, it uses `window.location.origin`.

**Q61: What keyboard shortcut triggers prediction in the web app?**

**A:** `Ctrl + Enter` while inside the SMILES input textarea.

**Q62: Can the application be used completely offline?**

**A:** Yes. Both backend and frontend run locally on `127.0.0.1:8000` without requiring an internet connection.

## PART 30 — REAL-WORLD DEBUGGING AND FAILURE SCENARIOS

---

### Scenario 1: Model Feature Count Mismatch

- **Symptom:** `ValueError: Number of features of the model must match the input. Model n_features is 2048, input n_features is 1024`.
- **Root Cause:** Backend was configured with `nBits=1024` while the serialized model was trained on `nBits=2048`.
- **Fix:** Ensure `metadata.json` stores `fingerprint_bits: 2048` and `backend/main.py` reads this value dynamically.

### Scenario 2: RDKit Parsing Crash on Special Characters

- **Symptom:** Backend returns HTTP 500 instead of HTTP 422 when user types `"C#C%99"`.
- **Root Cause:** Unhandled exception during fingerprint calculation.
- **Fix:** Check `if mol is None` immediately after `Chem.MolFromSmiles()` and raise `HTTPException(status_code=422)`.

### Scenario 3: CORS Policy Blocking Browser Requests

- **Symptom:** Browser console logs `Access to fetch at 'http://127.0.0.1:8000/predict' from origin 'null' has been blocked by CORS policy`.
- **Root Cause:** Frontend opened directly via `file:///` without backend CORS permissions.
- **Fix:** In `main.py`, include `CORSMiddleware` with `allow_origins=["*"]`.

### Scenario 4: Browser Fetch Fails Due to Server Offline

- **Symptom:** Frontend shows `"Could not reach the backend at http://127.0.0.1:8000"`.
- **Root Cause:** Uvicorn server is not running.
- **Fix:** Start the server via `uvicorn main:app --host 127.0.0.1 --port 8000 --reload`.

### Scenario 5: Missing Serialized Artifacts

- **Symptom:** Backend startup crashes with `RuntimeError: Failed to load model from ../best_model.pkl`.
- **Root Cause:** Running the backend before executing the training pipeline.
- **Fix:** Execute `python train_model.py` to generate `best_model.pkl` and `metadata.json`.

## PART 31 — SYSTEM LIMITATIONS

---

To maintain scientific integrity, clearly recognize the boundaries of this system:

1. **Target Specificity:** The model is trained **exclusively on human Cyclooxygenase-2 (COX-2)**. Predictions do not apply to COX-1, kinases, ion channels, or other enzymes.
  2. **2D Topological Nature:** Morgan circular fingerprints represent 2D graph connectivity. They do not model 3D stereochemistry, conformational flexibility, or enantiomer-specific binding differences.
  3. **Assay Noise:** The training dataset aggregates assays from multiple independent research papers with slight variations in laboratory protocols.
  4. **Applicability Domain:** The model predicts most accurately on drug-like organic small molecules. Predictions on inorganic compounds, organometallics, or macrocyclic peptides are out-of-domain and unreliable.
  5. **Not a Clinical Predictor:** A predicted `active` status indicates potential in vitro enzymatic binding; it does not establish safety, metabolic stability, or therapeutic efficacy in humans.
-

## PART 32 — PROJECT BOUNDARIES (What It Does vs. Does NOT Do)

---

### What the Project DOES:

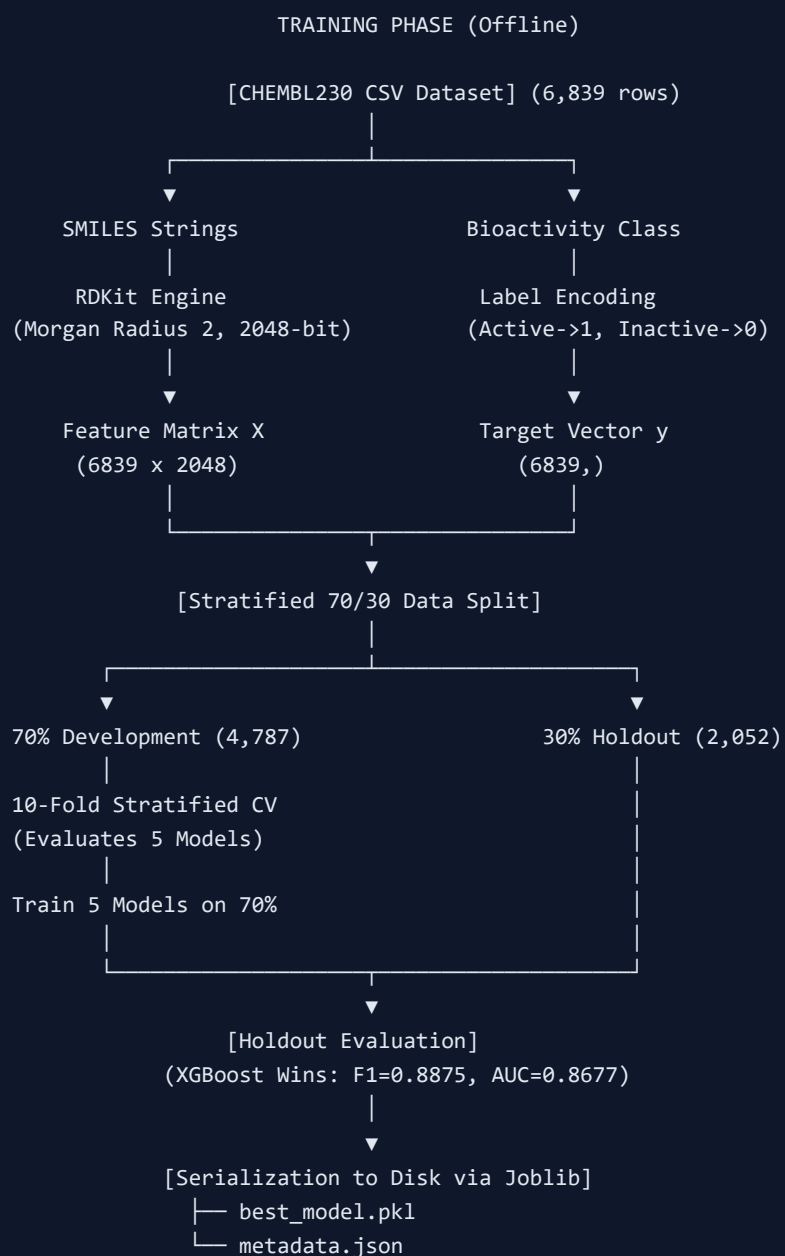
- Accepts valid chemical SMILES strings.
- Extracts reproducible 2048-bit Morgan circular fingerprints.
- Evaluates non-linear structural patterns via a tuned XGBoost classifier.
- Outputs active/inactive predictions with confidence probabilities.
- Provides a clean, modern REST API and responsive user interface for interactive virtual screening.

### What the Project DOES NOT Do:

- Does **not** perform 3D molecular docking or physics-based molecular dynamics simulations.
  - Does **not** predict drug toxicity, ADMET profiles, or metabolic clearance.
  - Does **not** replace physical laboratory bioassays or clinical trials.
  - Does **not** generate novel molecules from scratch (it is a discriminative classifier, not a generative model).
-

**PART 33 — FINAL COMPLETE MENTAL MODEL**

---



```
(XGBoost Evaluates)
|
Computes Class & Probability
|
Returns JSON Response
|
[Frontend Web UI]
|
Renders Result Badges
```

---

## PART 34 — RAPID REVISION & INTERVIEW CHEAT SHEET

### 1. 30-Second Elevator Pitch

"Our project is an end-to-end machine learning application for virtual drug screening. We trained and evaluated five classification algorithms on 6,839 COX-2 bioactivity records from the ChEMBL database using 2048-bit Morgan circular fingerprints. XGBoost achieved the best performance with an 88.75% holdout F1-score and 0.8677 ROC-AUC. We deployed this winning model via a production FastAPI REST backend and a responsive web UI that lets users test any chemical structure in real time."

### 2. 1-Minute Technical Summary

"We address the high cost of wet-lab screening by building a Ligand-Based Virtual Screening classifier for the human COX-2 enzyme. Using RDKit, candidate SMILES strings are converted into 2048-bit Morgan circular fingerprints (radius 2). We split the 6,839-sample ChEMBL dataset into a 70% development set for 10-fold stratified cross-validation and a 30% untouched holdout set. We compared XGBoost, Random Forest, Linear SVM, Naive Bayes, and RBF Networks. XGBoost outperformed all baselines, achieving 82.55% accuracy, 94.07% recall, 88.75% F1-score, and 0.8677 ROC-AUC. The pipeline automatically saves the best model using Joblib and serves it via an asynchronous FastAPI backend and a vanilla JavaScript frontend."

### 3. Top 10 Fundamental Facts

1. **Target:** Human Cyclooxygenase-2 (COX-2), ChEMBL ID `CHEMBL230`.
2. **Dataset Size:** 6,839 rows, 4 columns, 0 missing values.
3. **Class Ratio:** 5,004 Active (73.17%) and 1,835 Inactive (26.83%).
4. **Activity Threshold:**  $\text{IC}_{50} < 10,000 \text{ nM}$  is Active;  $\geq 10,000 \text{ nM}$  is Inactive.
5. **Feature Representation:** 2048-bit Morgan circular fingerprint (ECFP4, radius=2).
6. **Data Partition:** 70% Development (4,787), 30% Holdout (2,052), stratified.
7. **Validation:** 10-Fold Stratified Cross-Validation on the 70% development set.
8. **Winning Model:** XGBoost (`n_estimators=150`, `max_depth=6`, `learning_rate=0.15`).
9. **Winning Metrics:** Holdout F1: `0.8875`, ROC-AUC: `0.8677`, Recall: `0.9407`.
10. **Tech Stack:** Python, RDKit, Scikit-Learn, XGBoost, FastAPI, HTML5/CSS3/JavaScript.

### 4. Key Formulas

- **Precision:**  $\frac{TP}{TP + FP}$
- **Recall (Sensitivity):**  $\frac{TP}{TP + FN}$
- **Specificity:**  $\frac{TN}{TN + FP}$
- **F1-Score:**  $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
- **Paper AUC (Balanced Accuracy):**  $\frac{\text{Sensitivity} + \text{Specificity}}{2}$
- **Sigmoid Probability (SVM):**  $P(y=1) = \frac{1}{1 + e^{-\text{score}}}$